



EPC

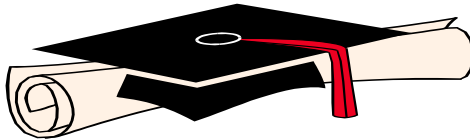
Elementary Programming with C

Semester: 1

The Exam Code: 01

Duration: 90 minutes

Total Marks: 20



Practical Examination Paper

Do not write on this question paper and return it to the Invigilator after the examination.
--

C Programming Exam: Data Management System

Total Points: 20

Duration: 90 minutes

Instructions

- Ensure your code is well-structured, easy to read, and includes appropriate comments explaining your logic.
 - Use standard C libraries and avoid external libraries.
 - Properly handle errors and edge cases.
-

Question 1: Basic Arithmetic and Error Handling (3 Points)

Task:

Write a program to calculate the average of three integers. Include input validation to ensure that all integers are positive.

Instructions:

1. Prompt the user to enter three integers.
2. Validate that the integers are positive. If not, display an error message and prompt again.
3. Calculate and display the average of the three integers.

Example Input/Output:

```
Enter three integers: 5 10 -3
Error: All numbers must be positive.
Enter three integers: 5 10 15
Average: 10.00
```

Points Distribution:

- Correct input handling: **1 Point**
 - Validation and error handling: **1 Point**
 - Correct calculation and output: **1 Point**
-

Question 2: Pointer and Array Manipulation (4 Points)

Task:

Write a program to reverse an array of integers using pointers.

Instructions:

1. Declare an array of size 10 and initialize it with user-provided integers.
2. Use a pointer to reverse the array in-place.
3. Display the original array and the reversed array.

Example Input/Output:

Enter 10 integers: 1 2 3 4 5 6 7 8 9 10

Original Array: 1 2 3 4 5 6 7 8 9 10

Reversed Array: 10 9 8 7 6 5 4 3 2 1

Points Distribution:

- Correct array initialization and input handling: **1 Point**
 - Proper use of pointers to reverse the array: **2 Points**
 - Displaying the original and reversed array: **1 Point**
-

Question 3: Structure and Data Management (6 Points)

Task:

Create a program to manage student records using an array of structures.

Instructions:

1. Define a structure `Student` with the following fields:
 - `id` (int): Student ID
 - `name` (char[50]): Student name
 - `marks` (float): Student marks
2. Write functions to:
 - Add a new student record.
 - Display all student records.
 - Calculate and display the average marks of all students.
3. Use an array of `Student` structures to store up to 5 students.

Example Input/Output:

Menu :

1. Add Student
2. Display All Students
3. Calculate Average Marks
4. Exit

Your choice: 1

Enter Student ID: 101

Enter Student Name: Alice

Enter Marks: 85

Your choice: 2

ID: 101, Name: Alice, Marks: 85.00

Your choice: 3

Average Marks: 85.00

Points Distribution:

- Correct structure definition: **1 Point**
 - Functionality for adding students: **2 Points**
 - Functionality for displaying all students: **2 Points**
 - Average marks calculation: **1 Point**
-

Question 4: Matrix Operations (4 Points)

Task:

Write a program to perform basic operations on a 3x3 matrix.

Instructions:

1. Input a 3x3 matrix from the user.
2. Write functions to:
 - Calculate the sum of all elements in the matrix.
 - Find the largest element in the matrix.
3. Display the sum and the largest element.

Example Input/Output:

Enter a 3x3 matrix:

1 2 3

```
4 5 6
7 8 9
```

Sum of all elements: 45

Largest element: 9

Points Distribution:

- Input and display the matrix: **1 Point**
 - Calculate the sum of elements: **1.5 Points**
 - Find the largest element: **1.5 Points**
-

Question 5: Dynamic Memory Allocation and Sorting (3 Points)

Task:

Write a program to sort an array of integers using dynamic memory allocation.

Instructions:

1. Dynamically allocate memory for an array of integers.
2. Accept the size of the array and its elements from the user.
3. Use a pointer-based sorting algorithm (e.g., bubble sort) to sort the array in ascending order.
4. Display the sorted array.

Example Input/Output:

```
Enter the number of elements: 5
```

```
Enter 5 integers: 10 5 8 3 6
```

```
Sorted Array: 3 5 6 8 10
```

Points Distribution:

- Correct use of dynamic memory allocation: **1 Point**
 - Implementing a pointer-based sorting algorithm: **1.5 Points**
 - Displaying the sorted array: **0.5 Points**
-

Submission Instructions

1. Submit all `.c` source code files.
2. Include comments explaining the logic for each major functionality.
3. Test your program thoroughly with valid and invalid inputs.
4. Package all files in a zipped folder named `C_Exam_<YourName>.zip`.